

Codex GPT-5.4 Reference vs Qwen2.5-Coder 14B Evaluation

Generated: 2026-06-19 12:35:22 UTC

Goal

This report evaluates the saved Qwen2.5-Coder 14B RAG answer run in docs/evaluations/answers/eval_v2_qwen2.5-14B-q4_K_M.json against the Codex GPT-5.4 reference answers stored in docs/evaluations/eval_answers_v2.json. The format follows the older codex_vs_local_llm_evaluation_20260423 report style, but the reference side here is the curated Codex V2 answer file rather than a new source-reading pass.

Test Environment

Codex Reference Side

- Model: GPT-5.4 (Codex reference answers)
- Reference file: docs/evaluations/eval_answers_v2.json
- Question set: docs/evaluations/eval_questions_v2.json
- Evaluation basis: semantic comparison against the Codex reference answer for each question

Local LLM Side

- Host: merlin-g-100.psi.ch
- Job / partition: 353390 on gwendolen
- Answer model: qwen2.5-coder:14b-instruct-q4_K_M
- Chunk explanation model: qwen2.5-coder:32b
- File / module / call-chain models: qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M
- Parser: tree_sitter
- Question count: 100
- Answer prompt mode: retrieval_answer_v2
- Mean answer latency: 6.95s
- Median answer latency: 6.60s

Retrieval Configuration Used by the Saved Run

- Candidate k: 20
- Supplementary k: 3
- Supplementary candidate k: 10
- Vector store chunk count: 4441
- Manifest embedding backend/model: ollama / nomic-embed-text

Important Caveat

The per-question verdicts are a structured comparison against the Codex reference answers, not an independent re-reading of every IPPL source file. A question is marked Correct when the local answer captures the central facts of the Codex reference; Partial when it contains relevant signal but misses important scope or implementation detail; and Incorrect when it abstains, points to the wrong subsystem, or omits the core reference answer.

Because the reference answers are intentionally concise, the grading emphasizes the main technical nouns, implementation locations, and algorithmic steps rather than exact wording. Ambiguous cases were treated conservatively as Partial rather than Correct.

Overall Result

	Metric		Value	
	---		---:	
	Questions		100	
	Correct		60	
	Partial		17	
	Incorrect		23	
	Average score (Correct=1, Partial=0.5, Incorrect=0)			0.685

Main Findings

- Strongest areas: Api Usage, Definition Location. These are mostly questions where the retrieved answer can be anchored to one well-described class, header, implementation file, or usage pattern.
- Weakest areas: Numerical Meaning, Parallelism And Kokkos. These questions require cross-file synthesis, precise implementation-location recovery, or careful numerical interpretation.
- The 14B run is slower than the 7B run, with mean latency 6.95s, but it often gives fuller file/symbol evidence.
- The most serious misses remain wrong-neighborhood answers: relevant-looking evidence from a neighboring subsystem can still displace the specific Codex reference answer.
- Compared with small-model behavior, the answers are generally more complete in file-purpose, API, and class-responsibility questions, while build/configuration and some numerical questions remain brittle.

Category Breakdown

	Category		Questions		Correct		Partial		Incorrect		Avg score	
	---		---:		---:		---:		---:		---	
	File Purpose		13		7		5		1		0.731	
	Definition Location		15		12		0		3		0.800	
	Class Responsibility		13		9		0		4		0.692	
	Algorithm		12		5		5		2		0.625	
	Data Flow		9		5		3		1		0.722	
	Api Usage		12		10		0		2		0.833	
	Parallelism And Kokkos		9		3		3		3		0.500	
	Boundary And Halo		5		3		1		1		0.700	
	Numerical Meaning		3		1		0		2		0.333	
	Build And Install		5		3		0		2		0.600	
	Testing And Workflow		4		2		0		2		0.500	

Detailed Per-Question Evaluation

File Purpose

file_001 - What does Ippl.h do?

- Verdict: Correct
- Codex reference answer: Declares the top-level IPPL runtime handles (Comm, Env, Info, Warn, Error) and the lifecycle wrappers initialize, finalize, fence, and abort.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.53
- Local answer excerpt: Answer: Ippl.h serves as a central header file in the root module of the

IPPL codebase. It provides essential functions and variables for managing application initialization, termination, communication, and error handling. Key symbols include `ippl::abort`,...

file_002 - What is the purpose of `Ippl.cpp`?

- Verdict: Partial
- Codex reference answer: Implements runtime setup/teardown: command-line option parsing, MPI communicator/environment creation, logger setup, Kokkos init/finalize, and abort/fence helpers.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: The `Ippl.cpp` file contains essential utility functions for managing initialization, finalization, and error handling within the IPPL framework. It includes functions like `abort`, `checkOption`, `getNumericalOption`, `fence`, `finalize`, and `init`...

file_003 - What does `FFT.hpp` implement?

- Verdict: Correct
- Codex reference answer: Implements `FFTBase` setup plus the FFT specialization family for complex/real, sine, cosine, and cosine-l transforms on IPPL fields via `heFFTe`.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.35
- Local answer excerpt: Answer: The `FFT.hpp` header file implements a base class template `FFTBase<Field, FFT, Backend, T>` and several specializations for Fast Fourier Transform (FFT) operations within the IPPL namespace. It supports various transform types such as real and comp...

file_004 - What is `BareField.h` responsible for?

- Verdict: Partial
- Codex reference answer: Defines the core distributed field container: Kokkos-backed storage, owned/allocated domains, field layout access, halo exchange hooks, and expression-style assignment.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.35
- Local answer excerpt: Answer: The `BareField.h` file defines the `ippl::BareField` class, which is a fundamental abstraction for representing fields in scientific computations within the IPPL framework. It includes methods to access various properties of the field such as `alloca`...

file_005 - What does `Field.h` add on top of `BareField`?

- Verdict: Correct
- Codex reference answer: Adds mesh awareness and boundary-condition management on top of `BareField`, plus volume integral/average helpers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.56
- Local answer excerpt: Answer: `ippl::Field.h` adds several functionalities and abstractions on top of `BareField`. Specifically, it provides a more comprehensive class for representing fields that includes methods for handling boundary conditions (`getFieldBC`, `setFieldBC`), `de`...

file_006 - What role does `HaloCells.h` play in IPPL?

- Verdict: Partial
- Codex reference answer: Defines the halo-exchange machinery for fields: pack/unpack, boundary exchange, and periodic handling for ghost cells.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.27
- Local answer excerpt: Answer: The HaloCells class and related structs within the IPPL library manage halo cells for field data, facilitating inter-process communication (IPC) in parallel computations. It includes functionalities like managing halo cell interactions, handling s...

file_007 - What is the purpose of FieldLayout.h?

- Verdict: Incorrect
- Codex reference answer: Defines how a global NDIndex domain is partitioned across MPI ranks, including local subdomains, neighbors, and communication ranges.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.14
- Local answer excerpt: Answer: FieldLayout.h defines the FieldLayout class and related helper functions within the ippl namespace. This class manages field distribution and domain information for scientific computations. It includes methods like getDomain, which provides ...

file_008 - What does ParticleBase.h define?

- Verdict: Partial
- Codex reference answer: Defines the generic particle container base class with positions, optional IDs, registered attributes, and a particle layout that controls migration.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: ParticleBase.h defines the ippl::ParticleBase class as a foundational abstraction for managing particle data within the IPPL framework. This class provides methods to handle attributes, layout, local and global particle counts, setting boundary ...

file_009 - What is the purpose of ParticleSpatialLayout.h?

- Verdict: Partial
- Codex reference answer: Defines the spatial particle layout that decides ownership by region and migrates particles to the correct rank after motion.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The ParticleSpatialLayout class, defined in ParticleSpatialLayout.h, manages spatial layout and communication regions for particles within the IPPL (Integrated Particle Physics Library). It provides methods such as getRegionLayout, fillHash,...

file_010 - What does ParticleAttrib.h implement?

- Verdict: Correct
- Codex reference answer: Implements the ParticleAttrib storage object: a Kokkos-view-backed particle attribute with resize, serialization, and mirror/access helpers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.39
- Local answer excerpt: Answer: ParticleAttrib.h implements a class template ippl::ParticleAttrib for managing and manipulating attributes associated with particles. It provides methods for serialization, memory management, and data retrieval such as deserialize, get_name,...

file_011 - What is UniformCartesian.h for?

- Verdict: Correct
- Codex reference answer: Defines the uniform Cartesian mesh class with spacing, origin, cell volume, total mesh volume, and vertex-position helpers.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.54
- Local answer excerpt: Answer: The UniformCartesian.h file defines the UniformCartesian class family in the ippl::Meshes namespace, specializing uniform Cartesian meshes. It provides methods for handling mesh properties such as vertex positions, cell volumes, mesh spacing, ...

file_012 - What does OrthogonalRecursiveBisection.h do?

- Verdict: Correct
- Codex reference answer: Declares the orthogonal recursive bisection domain-decomposition/load-balancing class and its cut/median/repartition helpers.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.47
- Local answer excerpt: Answer: The file OrthogonalRecursiveBisection.h defines the class ippl::OrthogonalRecursiveBisection, which is used for decomposing computational domains using an orthogonal recursive bisection algorithm. This class includes methods such as cutDomain,...

file_013 - What is the purpose of CIC.h in the Interpolation subsystem?

- Verdict: Correct
- Codex reference answer: Declares the first-order cloud-in-cell interpolation helpers for scatter/gather between particles and grid fields.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The CIC.h file in the Interpolation subsystem defines function declarations for handling Cloud-in-Cell (CIC) interpolation methods within the IPPL codebase. These functions manage operations such as scattering particles' properties to a field (sc...

Definition Location

loc_001 - Where is FFT initialization implemented in IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT initialization lives in src/FFT/FFT.hpp, especially FFTBase construction/setup and the transform-specific constructors that build the heFFTe plans.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.30
- Local answer excerpt: Answer: The retrieved context does not provide specific information on where FFT initialization is implemented in the IPPL codebase. The available chunks discuss general initialization functions and methods for various components such as mpi::Environme...

loc_002 - Where does IPPL set up the MPI environment?

- Verdict: Incorrect

- Codex reference answer: IPPL sets up MPI in src/lppl.cpp::initialize() by constructing mpi::Environment and mpi::Communicator; the low-level MPI_Init check/call is in src/Communicate/Environment.cpp.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.19
- Local answer excerpt: Answer: The retrieved context does not provide specific information on where IPPL sets up the MPI environment. The provided chunks discuss FFT setup and field initialization but do not mention MPI environment configuration. Evidence: - data/raw/ippl/src/FFT...

loc_003 - Where is Kokkos initialized and finalized inside IPPL?

- Verdict: Correct
- Codex reference answer: Kokkos is initialized and finalized directly in src/lppl.cpp inside ippl::initialize() and ippl::finalize().
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.92
- Local answer excerpt: Answer: Kokkos is initialized inside the ippl::initialize function in lppl.cpp, which takes command-line arguments (argc and argv) and an MPI communicator. It is finalized in the same file within the ippl::finalize function. Evidence: - src/lppl.c...

loc_004 - Where is the orthogonal recursive bisection load balancer implemented?

- Verdict: Correct
- Codex reference answer: The ORB load balancer is implemented in src/Decomposition/OrthogonalRecursiveBisection.hpp with declarations in the matching header.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.71
- Local answer excerpt: Answer: The orthogonal recursive bisection load balancer is implemented in the OrthogonalRecursiveBisection.h header file within the src/Decomposition directory. The class ippl::OrthogonalRecursiveBisection provides methods for decomposing computation...

loc_005 - Where are boundary conditions applied to a Field?

- Verdict: Correct
- Codex reference answer: Field BCs are attached in Field::setFieldBC() and actually applied through BConds::apply() and the concrete face classes in BcTypes.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: Boundary conditions for a Field are applied through methods like applyBCs and setPeriodicBoundaryConditions in the FDTDsSolverBase class. The applyBCs method likely applies general boundary conditions using dot products and field data, while ...

loc_006 - Where is the CIC (cloud-in-cell) scatter implemented?

- Verdict: Correct
- Codex reference answer: CIC scatter is implemented in src/Interpolation/CIC.hpp, primarily scatterToPoint() and scatterToField().
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.82

- Local answer excerpt: Answer: The CIC (Cloud-in-Cell) scatter implementation is primarily found within the `ippl::detail::scatterToField` function declared and defined in the `CIC.h` and `CIC.hpp` files. This function is responsible for scattering particle data into fields using...

loc_007 - Where is the CIC gather implemented?

- Verdict: Correct

- Codex reference answer: CIC gather is implemented in `src/Interpolation/CIC.hpp`, primarily `gatherFromPoint()` and `gatherFromField()`.

- Comparison: The answer names the relevant implementation location and captures the core reference fact.

- Reference-token coverage: 0.82

- Local answer excerpt: Answer: The CIC gather implementation is located within the `CIC.hpp` file. This file defines the methods for interpolation operations, specifically `scatterToField`, `scatterToPoint`, `gatherFromField`, and `gatherFromPoint`. The gathering process involves...

loc_008 - Where is halo exchange for BareField implemented?

- Verdict: Correct

- Codex reference answer: Halo exchange for `BareField` is surfaced in `src/Field/BareField.hpp` and actually performed in `src/Field/HaloCells.hpp`.

- Comparison: The answer names the relevant implementation location and captures the core reference fact.

- Reference-token coverage: 0.69

- Local answer excerpt: Answer: Halo exchange for `BareField` is implemented in the `BareField.hpp` file through methods such as `accumulateHalo` and `fillHalo`. These methods manage the communication of halo (boundary) data between different parts of a decomposed field across mul...

loc_009 - Where are the FFT-based open-boundary Poisson solver routines defined?

- Verdict: Correct

- Codex reference answer: The FFT open-boundary Poisson routines are defined in `src/PoissonSolvers/FFTOpenPoissonSolver.h` and `src/PoissonSolvers/FFTOpenPoissonSolver.hpp`.

- Comparison: The answer names the relevant implementation location and captures the core reference fact.

- Reference-token coverage: 0.83

- Local answer excerpt: Answer: The FFT-based open-boundary Poisson solver routines are defined in the `FFTOpenPoissonSolver.hpp` and `FFTOpenPoissonSolver.h` files within the `src/PoissonSolvers/` directory of the IPPL codebase. Evidence: - `data/raw/ippl/src/PoissonSolvers/FFTOpe...`

loc_010 - Where is the conjugate gradient solver algorithm implemented?

- Verdict: Correct

- Codex reference answer: The CG/PCG algorithm implementation is in `src/LinearSolvers/PCG.h`; the Poisson wrapper that instantiates it is `src/PoissonSolvers/PoissonCG.h`.

- Comparison: The answer names the relevant implementation location and captures the core reference fact.

- Reference-token coverage: 0.67

- Local answer excerpt: Answer: The conjugate gradient solver algorithm is implemented in the `PoissonCG::solve` method within the `PoissonSolvers/PoissonCG.h` file. This method sets up and executes a Poisson equation solver using the Conjugate Gradient (CG) method. Evidence: - sr...

loc_011 - Where is the FEM Poisson solver implemented?

- Verdict: Correct
- Codex reference answer: The FEM Poisson solver is implemented in `src/PoissonSolvers/FEMPoissonSolver.h`.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.83
- Local answer excerpt: Answer: The FEM Poisson solver is implemented in the `FEMPoissonSolver.h` file within the `src/PoissonSolvers` directory. The class `ippl::FEMPoissonSolver` inherits from `Poisson<FieldLHS, FieldRHS>` and uses finite element methods (FEM) combined with the ...

loc_012 - Where is the FDTD Maxwell solver base class defined?

- Verdict: Incorrect
- Codex reference answer: The FDTD Maxwell solver base class is defined in `src/MaxwellSolvers/FDTSolverBase.h` with implementation in `FDTSolverBase.hpp`.
- Comparison: The answer cites a different implementation location than the Codex reference.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: The FDTD Maxwell solver base class is defined in the `FieldSolverBase.h` file within the `ippl::Manager` namespace. Evidence: - `data/raw/ippl/src/Manager/FieldSolverBase.h` : `FieldSolverBase` — abstract base class for field solvers, providing common f...

loc_013 - Where is the LagrangeSpace class defined?

- Verdict: Correct
- Codex reference answer: The `LagrangeSpace` class is defined in `src/FEM/LagrangeSpace.h` with methods implemented in `src/FEM/LagrangeSpace.hpp`.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: The `LagrangeSpace` class is defined in the file `LagrangeSpace.h` within the `ippl` namespace. Evidence: - `data/raw/ippl/src/FEM/LagrangeSpace.h` : `LagrangeSpace` — core finite element space class for Lagrangian elements, handling mesh-based computat...

loc_014 - Where is the particle update routine (that moves particles across ranks) implemented?

- Verdict: Correct
- Codex reference answer: Particle migration between ranks is implemented in `ParticleSpatialLayout::update()` in `src/Particle/ParticleSpatialLayout.hpp`.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.70
- Local answer excerpt: Answer: The particle update routine that moves particles across ranks is implemented in both `ParticleSpatialLayout.hpp` and `ParticleSpatialOverlapLayout.hpp`. In `ParticleSpatialLayout.hpp`, the method `ippl::ParticleSpatialLayout<T, Dim, Mesh, Properties...`

loc_015 - Where are IPPL timers defined?

- Verdict: Correct
- Codex reference answer: IPPL timers are defined by the singleton timing layer in `src/Utility/IpplTimings.h/.cpp`, not just the low-level `Timer` helper.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.

- Reference-token coverage: 0.44
- Local answer excerpt: Answer: IPPL timers are defined in the IpplTimings.h header file. The class IpplTimerInfo is used to store timer values, and functions like startTimer are implemented in IpplTimings.cpp to manage these timers. Evidence: - src/Utility/IpplTimings.h :...

Class Responsibility

cls_001 - What is the main responsibility of the BareField class?

- Verdict: Correct
- Codex reference answer: BareField is the core distributed field storage class: it owns the data view, domain metadata, layout link, and halo operations.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The BareField class is a fundamental abstraction for representing fields in scientific computations within the IPPL framework. It manages memory layout and access patterns for multidimensional fields, including handling ghost cells for parallel co...

cls_002 - What is the main responsibility of the Field class compared to BareField?

- Verdict: Incorrect
- Codex reference answer: Field extends BareField with a mesh, boundary-condition container, and mesh-aware integral/average semantics.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.19
- Local answer excerpt: Answer: The provided context does not contain information about a Field class or its responsibilities compared to BareField. The context only discusses the BareField class, which is described as managing memory and access patterns for multidimensional...

cls_003 - What does the FieldLayout class do and what are its main template parameters?

- Verdict: Correct
- Codex reference answer: FieldLayout<Dim> distributes a global NDIndex<Dim> over ranks and stores local domains, neighbors, and communication ranges; its only template parameter is Dim.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.47
- Local answer excerpt: Answer: The FieldLayout class manages field distribution and domain details within IPPL. It handles partitioning of a multi-dimensional index space (domain) across MPI ranks, allowing specific dimensions to be marked as non-partitioned ('SERIAL'). The...

cls_004 - What is the purpose of the ParticleBase class and how is it meant to be used?

- Verdict: Correct
- Codex reference answer: ParticleBase is the user-extensible base container for particles; derived classes add attributes and register them so layout-driven migration can move them.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.45
- Local answer excerpt: Answer: The ParticleBase class serves as a foundational container and manager for particles in the IPPL (Integrated Plasma Physics Library). It manages particle data attributes, handles particle creation and destruction, and interfaces with a layout mecha...

cls_005 - What does the ParticleSpatialLayout class do?

- Verdict: Correct
- Codex reference answer: ParticleSpatialLayout decides which spatial region/rank owns each particle and performs the migration/update workflow after particles move.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.39
- Local answer excerpt: Answer: The ParticleSpatialLayout class manages spatial layout and communication regions for particles within the IPPL codebase. It provides methods to configure, update, and retrieve spatial information necessary for particle distribution and neighbor in...

cls_006 - What is the responsibility of UniformCartesian?

- Verdict: Correct
- Codex reference answer: UniformCartesian represents a uniform Cartesian mesh and exposes spacing/origin/volume and vertex-position geometry queries.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The UniformCartesian class defines uniform Cartesian meshes within the IPPL codebase. It inherits from Mesh<T, Dim> and provides methods to manage mesh properties such as vertex positions, cell volume, mesh spacing, and overall mesh volume. Key ...

cls_007 - What does the Mesh base class define?

- Verdict: Incorrect
- Codex reference answer: Mesh is the abstract base interface for meshes: origin, grid size, spacing, vertex positions, and volume queries.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.13
- Local answer excerpt: I'm sorry, but the provided retrieved context does not contain any information about a Mesh base class or its definitions. The context includes details about other types and classes such as _Variant_base, _Move_ctor_base, Tuple, Verifier, etc., bu...

cls_008 - What is the responsibility of the Communicator class in ippl::mpi?

- Verdict: Correct
- Codex reference answer: ippl::mpi::Communicator wraps MPI point-to-point, collectives, tag handling, and managed communication buffers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.46
- Local answer excerpt: Answer: The Communicator class in ippl::mpi manages MPI communicators and their associated operations. It provides methods for managing MPI communicators such as barriers, rank retrieval, size determination, and buffer handling. Key functions include abo...

cls_009 - What is the role of the Environment class in ippl::mpi?

- Verdict: Correct
- Codex reference answer: ippl::mpi::Environment manages MPI environment lifetime checks/teardown and provides an abort hook.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.40
- Local answer excerpt: Answer: The Environment class in ippl::mpi manages MPI environment setup and teardown. It includes constructors for initializing MPI, destructors for finalizing MPI, and utility functions like initialized and finalized to check the status of the MPI...

cls_010 - What does the Archive class do in the Communicate module?

- Verdict: Incorrect
- Codex reference answer: Archive serializes and deserializes Kokkos views and vector-valued views into raw byte buffers for MPI transfer.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.08
- Local answer excerpt: Answer: The question "What does the Archive class do in the Communicate module?" cannot be answered based on the provided retrieved context. The context does not contain any information about an Archive class within the Communicate module. Evidence: - N...

cls_011 - What is the role of the BConds class?

- Verdict: Correct
- Codex reference answer: BConds is the container of boundary-condition faces and orchestrates neighbor discovery, BC application, and ghost-to-physical assignment.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: The ippl::BConds class is responsible for handling boundary conditions in fields within the IPPL (Integrated Plasma Physics Library). It encapsulates various operations related to boundary conditions, including applying them, managing ghost cells,...

cls_012 - What is the role of the BaseManager class?

- Verdict: Correct
- Codex reference answer: BaseManager provides the simulation loop skeleton: pre_run, pre_step, advance, post_step, and run(nt).
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.87
- Local answer excerpt: Answer: The BaseManager class serves as a foundational abstract class within the IPPL namespace, defining essential methods such as run, pre_run, post_step, and advance. It acts as a base class for managing simulation workflows, providing a struct...

cls_013 - What does the Index class represent and what operations does it support?

- Verdict: Incorrect
- Codex reference answer: Index represents a regular strided integer range/slice and supports range-style arithmetic and comparison operations used in domain descriptions.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.12
- Local answer excerpt: Answer: The retrieved context does not provide information about an Index class or its operations. The context discusses various other classes such as Tuple, _Build_FUN, Verifier, _Uninitialized, and Min, but does not mention an Index clas...

Algorithm

alg_001 - How does the FFT-based open-boundary Poisson solver compute the electrostatic

potential?

- Verdict: Partial
- Codex reference answer: The FFT open-boundary solver embeds/extends the RHS onto larger domains, FFTs both the density and a precomputed Green's function, multiplies in Fourier space, inverse transforms, and optionally computes gradients/Hessians.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.41
- Local answer excerpt: Answer: The FFT-based open-boundary Poisson solver computes the electrostatic potential by utilizing Fast Fourier Transform (FFT) techniques as part of its core methodology. The class `ippl::FFTOpenPoissonSolver` is specialized to handle this computation, l...

alg_002 - How does the FFT-based periodic Poisson solver work in IPPL?

- Verdict: Correct
- Codex reference answer: The periodic FFT solver FFTs the charge density, divides each Fourier mode by $|k|^2$ (with the zero mode handled separately), then inverse transforms; gradients are computed by multiplying by $ik/|k|^2$ before the inverse FFT.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.45
- Local answer excerpt: Answer: The FFT-based periodic Poisson solver in IPPL uses Fast Fourier Transforms to solve the periodic Poisson problem. It involves transforming the charge density field into frequency space using FFTs, multiplying by a precomputed Green's function, and t...

alg_003 - How does the truncated Green periodic Poisson solver differ from the standard periodic one?

- Verdict: Correct
- Codex reference answer: The truncated-Green periodic solver replaces the simple $1/|k|^2$ kernel with the FFT of a truncated/erf-smoothed Green's function $\text{forceConstant} * \text{erf}(\alpha r) / r$, then follows the same transform-multiply-inverse pattern.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.42
- Local answer excerpt: Answer: The truncated Green periodic Poisson solver (`FFTTruncatedGreenPeriodicPoissonSolver`) differs from the standard periodic one (`FFTPeriodicPoissonSolver`) primarily in how they handle the Green's function and potentially mesh spacing updates. The tr...

alg_004 - How does IPPL's CG solver iterate in the PCG.h implementation?

- Verdict: Incorrect
- Codex reference answer: In `PCG.h`, the CG iteration forms residual/direction fields, applies the operator, computes alpha and beta from inner products, updates the solution/residual/direction, and stops on tolerance or iteration count; the preconditioned variant inserts M^{-1} applications on the residual.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.15
- Local answer excerpt: Answer: IPPL's CG solver iterates through a series of steps within the PCG class template and its specialization. The iteration process is not explicitly detailed in the provided context, but it is clear that the solver involves setting up operators, init...

alg_005 - How is cloud-in-cell interpolation performed in IPPL (both scatter and gather

directions)?

- Verdict: Incorrect
- Codex reference answer: CIC computes lower/upper weights per axis, scatters with atomic adds to the 2^{Dim} neighboring grid points, and gathers by summing the same weighted stencil back to particles.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.09
- Local answer excerpt: Answer: In IPPL, cloud-in-cell (CIC) interpolation for both scatter and gather directions is handled within the ParticleAttrib class template. The scatter method distributes particle attributes from a ParticleAttrib object onto a field (Field& f) us...

alg_006 - How does the standard FDTD solver advance the electromagnetic field in time?

- Verdict: Partial
- Codex reference answer: The standard FDTD solver advances the source-like field with a second-order finite-difference update using the previous and current time levels plus neighboring cells and source terms, then fills halos and reapplies BCs.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The standard FDTD solver advances the electromagnetic field in time by updating the fields iteratively within the step() method. This method uses a leapfrog scheme with anisotropic coefficients to compute new field values based on previous and cur...

alg_007 - How does the LagrangeSpace class evaluate degrees of freedom per element?

- Verdict: Correct
- Codex reference answer: LagrangeSpace evaluates element DOFs by mapping local element indices to global DOF indices, then evaluates order-1 reference-element shape functions and gradients on those local DOFs.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.32
- Local answer excerpt: Answer: The LagrangeSpace class evaluates degrees of freedom per element through the getLagrangeNumElementDOFs function, which calculates the number based on the dimension and order specified during instantiation. The LagrangeSpace class inherits from ...

alg_008 - How is Gauss-Jacobi quadrature implemented and used in IPPL FEM?

- Verdict: Correct
- Codex reference answer: Gauss-Jacobi quadrature computes nodes/weights by Newton iteration on Jacobi polynomial roots, starting from Chebyshev or LehrFEM-style guesses, and uses the resulting rule on reference elements in FEM.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.54
- Local answer excerpt: Answer: The Gauss-Jacobi quadrature is implemented through the GaussJacobiQuadrature class, which inherits from the Quadrature base class. The GaussJacobiQuadrature class defines parameters alpha_m, beta_m, and iteration limits for Newton methods ...

alg_009 - How does the orthogonal recursive bisection decide where to cut the domain?

- Verdict: Partial

- Codex reference answer: ORB chooses a cut axis from the domain geometry and then finds a weight-balanced cut/median so recursive splits distribute load more evenly.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.37
- Local answer excerpt: Answer: The orthogonal recursive bisection decides where to cut the domain by first identifying the longest axis of the field layout using the findCutAxis method. It then computes the median along this identified axis using the findMedian method. The do...

alg_010 - How does inverse transform sampling generate particles from a given distribution in IPPL?

- Verdict: Partial
- Codex reference answer: Inverse transform sampling maps uniform random values in CDF space to samples by estimating the inverse and refining it with Newton-Raphson, dimension by dimension.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.35
- Local answer excerpt: Answer: Inverse transform sampling generates particles by transforming uniformly distributed random numbers into samples that follow a specific target distribution. The generate method in ippl::random::InverseTransformSampling fills a provided view with...

alg_011 - How does the FFT class interface with heFFTe to perform complex-to-complex and real-to-complex transforms?

- Verdict: Partial
- Codex reference answer: The FFT layer builds heFFTe boxes/plans in setup(), copies IPPL field data into the required views/subviews, calls heFFTe forward/backward transforms, and copies results back for CC/RC/DST/DCT variants.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.31
- Local answer excerpt: Answer: The FFT.h header file defines a family of Fast Fourier Transform (FFT) classes within the IPPL library, including both base and specialized variants. The FFT class interfaces with heFFTe by converting domain bounds into Heffte-compatible boxes (b...

alg_012 - How does IPPL's preconditioned CG use the preconditioner defined in Preconditioner.h?

- Verdict: Correct
- Codex reference answer: PCG applies a chosen preconditioner from Preconditioner.h to the residual, uses the preconditioned residual in its inner products and direction updates, and iterates until tolerance/max iterations.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.28
- Local answer excerpt: Answer: IPPL's preconditioned Conjugate Gradient (CG) solver uses the preconditioner defined in Preconditioner.h by setting up a specific preconditioning strategy through the setPreconditioner method. This method configures the CG solver with a chosen p...

Data Flow

flow_001 - How does charge density flow from particles to the grid in a typical PIC step in IPPL?

- Verdict: Correct
- Codex reference answer: In the PIC workflow, particle charges and positions are deposited onto the grid with CIC (scatter) or FEM assembly (assemble_rhs_from_particles), accumulated into rho, and then normalized/background-shifted in the manager before solving.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.42
- Local answer excerpt: Answer: Charge density flow from particles to the grid in a typical PIC step in IPPL is handled by the assemble_rhs_from_particles function. This function deposits weighted values from particles into a field based on their positions using shape functions ...

flow_002 - How does the electric field flow from the grid back to particles after a Poisson solve?

- Verdict: Correct
- Codex reference answer: After the field solve, the manager gathers the electric field back to particles with CIC gather(...) or FEM interpolate_grad_to_diracs(...), storing particle-local E values for the push step.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.26
- Local answer excerpt: Answer: The electric field is computed after solving the Poisson equation by either: 1. If isGradFD_m and the output type (out) is Base::SOL_AND_GRAD, the electric field is directly calculated as the gradient of the potential using finite differences:...

flow_003 - How are halo cells exchanged between neighboring MPI ranks for a BareField?

- Verdict: Incorrect
- Codex reference answer: Halo exchange uses FieldLayout neighbor/range metadata to pack field subviews, send/receive them with MPI, unpack into ghost or physical regions, and handle serial-periodic dimensions locally.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.19
- Local answer excerpt: Answer: The exchange of halo cells between neighboring MPI ranks for a BareField is managed through methods defined within the HaloCells.h and HaloCells.hpp headers. Specifically, the HaloCells class provides functionalities such as exchangeBou...

flow_004 - How are particles migrated between MPI ranks by ParticleSpatialLayout?

- Verdict: Partial
- Codex reference answer: ParticleSpatialLayout::update() applies particle BCs, locates destination ranks, advertises receive counts with RMA, sends all registered attributes, destroys invalid local particles, and receives the incoming ones.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.23
- Local answer excerpt: Answer: The ParticleSpatialLayout class manages spatial layout and communication regions for particles. It includes methods like updateLayout, which is crucial for migrating particles between MPI ranks. However, the specific details of how particles are...

flow_005 - How does FieldLayout decide which index ranges live on which rank?

- Verdict: Correct

- Codex reference answer: FieldLayout uses the partitioner and the requested parallel/serial decomposition flags to assign each rank a local NDIndex, plus neighbor and communication range tables.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The FieldLayout class manages field distribution and domain details, including which index ranges live on which rank. It uses methods like addNeighbors, calcWidths, findNeighbors, and getBounds to determine neighbor relationships and calcu...

flow_006 - How is a ParameterList used to configure the FFT and solver classes?

- Verdict: Correct
- Codex reference answer: ParameterList is merged into FFT and solver objects, then consumed by setup()/initialize()/solve() for communication mode, pencil/reorder flags, transform direction, solver type, tolerances, preconditioner settings, and output type.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.28
- Local answer excerpt: Answer: A ParameterList is used to configure the FFT and solver classes by providing necessary parameters for their initialization. In the FFT.h file, the constructor of the FFT<RCTransform, RealField> class takes a const ParameterList& params as an...

flow_007 - How does data flow through the BaseManager's pre_step / advance / post_step hooks during a simulation?

- Verdict: Partial
- Codex reference answer: BaseManager::run() executes pre_step(), advance(), and post_step() for each timestep; derived managers use those hooks for deposit/solve/gather, time updates, and output.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.48
- Local answer excerpt: Answer: The BaseManager class defines essential methods for managing the execution flow of simulations, including pre_step, advance, and post_step. However, the provided context does not specify how these hooks interact with each other or provide a ...

flow_008 - How does the FEM solver move data between fields and FEMVector representations?

- Verdict: Correct
- Codex reference answer: FEM solvers move data by assembling field data into FEM load/stiffness representations (evaluateLoadVector, evaluateAx, FEMVector) and then interpolating the solved field quantities back where needed.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.25
- Local answer excerpt: Answer: The FEM solver moves data between fields and FEMVector representations primarily through halo exchange mechanisms. The accumulateHalo method within the FEMVector<T> class handles this process by packing local data into communication buffers (...)

flow_009 - How does OrthogonalRecursiveBisection redistribute particles after rebalancing?

- Verdict: Partial
- Codex reference answer: After ORB recomputes the decomposition, particle ownership changes are

realized by updating the layout/regions and then migrating particles with the usual particle update path.

- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.

- Reference-token coverage: 0.28

- Local answer excerpt: Answer: The OrthogonalRecursiveBisection class is responsible for redistributing particles after rebalancing. It uses an orthogonal recursive bisection algorithm to decompose computational domains and balance the load across MPI ranks. The key methods inv...

Api Usage

api_001 - How do I create a 3D Field of doubles in IPPL?

- Verdict: Incorrect

- Codex reference answer: A typical 3D field is `Field<double, 3, UniformCartesian<double, 3>, Cell>` constructed from a `UniformCartesian` mesh and a `FieldLayout<3>`.

- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.

- Reference-token coverage: 0.07

- Local answer excerpt: The provided context does not contain specific information on how to create a 3D Field of doubles in IPPL. The chunks primarily discuss initialization functions and variant type implementations within the library. Evidence: - data/raw/ippl/src/Ippl.cpp : ip...

api_002 - How do I construct a UniformCartesian mesh with custom spacing and origin?

- Verdict: Correct

- Codex reference answer: Construct the mesh as `UniformCartesian<double, Dim> mesh(domain, hx, origin);` where `hx` is the spacing vector and `origin` is the mesh origin.

- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.62

- Local answer excerpt: Answer: To construct a `UniformCartesian` mesh with custom spacing and origin in IPPL, you use the constructor of the `ippl::UniformCartesian` class. The constructor takes three parameters: an `NDIndex<Dim>` object representing the number of grid points alo...

api_003 - How do I attach a boundary condition (e.g. periodic) to a Field?

- Verdict: Incorrect

- Codex reference answer: Create a `BConds<FieldT, Dim>`, assign per-face BC objects such as `PeriodicFace<FieldT>`, then call `field.setFieldBC(bc)`.

- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.

- Reference-token coverage: 0.19

- Local answer excerpt: Answer: To attach a periodic boundary condition to a `Field` in IPPL, you can use the `setPeriodicBoundaryConditions` method defined within the `FDTDSolverBase` class template. This method sets up periodic boundary conditions for the electromagnetic fields us...

api_004 - How do I define a custom particle class that adds mass and velocity attributes on top of ParticleBase?

- Verdict: Correct

- Codex reference answer: Define a class derived from `ParticleBase<ParticleSpatialLayout<...>>`, add mass and velocity attributes as members, and register them with `addAttribute()` in the constructor.

- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.53
- Local answer excerpt: Answer: To define a custom particle class that adds mass and velocity attributes on top of ParticleBase, you need to inherit from ParticleBase and use its method addAttribute to add new attributes. Here is an example of how you can define such a class...

api_005 - How do I call ippl::initialize and ippl::finalize in a main program?

- Verdict: Correct
- Codex reference answer: Call the free namespace functions ippl::initialize(argc, argv); and ippl::finalize(); from main, as shown in the docs/examples.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.53
- Local answer excerpt: Answer: To call ippl::initialize and ippl::finalize in a main program, you need to pass the command-line arguments (argc, argv) and an MPI communicator. Here is how you can do it: `cpp #include <mpi.h> #include "Ippl.h" int main(int argc, char* ar...`

api_006 - How do I set up and run a forward FFT on an IPPL Field?

- Verdict: Correct
- Codex reference answer: Set up the relevant layouts/fields/parameters, build an FFT<CCTransform, ...> or FFT<RCTransform, ...> object, and call transform(FORWARD, ...) on the field(s).
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.53
- Local answer excerpt: Answer: To set up and run a forward FFT on an IPPL Field, you need to create an instance of the FFT class template specialized for your transform type (e.g., RCTransform, CCTransform) and field type (RealField, ComplexField). The constructor of th...

api_007 - How do I construct an FFTPeriodicPoissonSolver and solve for the potential?

- Verdict: Correct
- Codex reference answer: Construct FFTPeriodicPoissonSolver(lhs, rhs) (or default construct then set fields/params), ensure the RHS is attached, and call solve() to write the potential/gradient outputs.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.61
- Local answer excerpt: Answer: To construct an FFTPeriodicPoissonSolver and solve for the potential, you need to follow these steps: 1. Include the necessary headers: Ensure you include FFTPeriodicPoissonSolver.h and any other dependencies listed in the retrieved context....

api_008 - How do I invoke the conjugate gradient Poisson solver on a Field?

- Verdict: Correct
- Codex reference answer: Use PoissonCG<FieldT> (or the Alpine FieldSolver wrapper around it), set its fields/parameters, and call solve() so the internal CG/PCG object iterates on the field.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.35
- Local answer excerpt: Answer: To invoke the conjugate gradient Poisson solver on a Field in IPPL, you need to create an instance of the PoissonCG class and set up the left-hand side (lhs) and right-hand side (rhs) fields. The solve method of the PoissonCG class is th...

api_009 - How do I sample particles from a normal distribution using IPPL Random utilities?

- Verdict: Correct
- Codex reference answer: For Gaussian draws, IPPL exposes both randn convenience generation and the more general InverseTransformSampling + NormalDistribution path.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: To sample particles from a normal distribution using IPPL's Random utilities, you can use the InverseTransformSampling class. This class leverages the cumulative distribution function (CDF) of a given distribution to generate random samples. S...

api_010 - How do I register particle attributes with ParticleBase so that they are communicated on migration?

- Verdict: Correct
- Codex reference answer: Register each particle attribute with addAttribute(...); migration then serializes and transfers all registered attributes automatically.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: To register particle attributes with ParticleBase for communication during migration, you need to define a derived class from ParticleBase, instantiate ParticleAttribute<T> objects for each attribute, and add them using the addAttribute meth...

api_011 - How do I access the MPI rank and size through the IPPL Communicator?

- Verdict: Correct
- Codex reference answer: Access rank and size with ippl::Comm->rank() and ippl::Comm->size() (or the same methods on a Communicator object).
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: To access the MPI rank and size through the IPPL Communicator, you can use the methods Communicator::rank and Communicator::size, respectively. These methods are part of the ippl::mpi::Communicator class defined in Communicator.h. Evidence: ...

api_012 - How do I use IpplTimings to measure a region of code?

- Verdict: Correct
- Codex reference answer: Get a timer handle with IpplTimings::getTimer("name"), surround the code with startTimer() / stopTimer(), and print with IpplTimings::print().
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.71
- Local answer excerpt: Answer: To use IpplTimings to measure a region of code, follow these steps: 1. Create or retrieve a timer reference: cpp IpplTimings::TimerRef val = IpplTimings::getTimer("timer name"); 2. Start the timer: cpp IpplTimings::startTimer(val); 3...

Parallelism And Kokkos

par_001 - How does IPPL achieve performance portability across CPUs and GPUs?

- Verdict: Partial
- Codex reference answer: IPPL gets performance portability by combining Kokkos execution/memory abstractions with MPI and heFFTe, so the same code paths can target CPUs and GPUs.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope,

file, or algorithm details.

- Reference-token coverage: 0.41
- Local answer excerpt: IPPL achieves performance portability across CPUs and GPUs through its integration with Kokkos. The `ippl::initialize` function sets up the environment for MPI parallel processing and initializes Kokkos with command-line arguments, enabling it to manage bot...

par_002 - Why does IPPL use Kokkos::View for field storage?

- Verdict: Incorrect
- Codex reference answer: IPPL uses Kokkos::View because fields need execution-space-aware storage, subviews, mirrors, and portable kernels across host/device memory spaces.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.27
- Local answer excerpt: Answer: IPPL uses Kokkos::View for field storage. The context provided does not specify why IPPL chose Kokkos::View over other data structures, so it is not possible to answer this question based solely on the given information. Evidence: - `src/lppl.cpp` : i...

par_003 - How does IPPL parallelize loops over field indices using Kokkos RangePolicy?

- Verdict: Correct
- Codex reference answer: Loops are parallelized through `ippl::getRangePolicy` / `createRangePolicy` and the `ippl::parallel_for` wrapper over Kokkos RangePolicy/MDRangePolicy.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: IPPL parallelizes loops over field indices using the `createRangePolicy` method from the `ParallelDispatch.h` file. This method constructs a range policy based on provided begin and end indices for multi-dimensional spaces. It returns a policy objec...

par_004 - How does IPPL support mixed precision (float vs double) for fields?

- Verdict: Incorrect
- Codex reference answer: Precision is chosen by the field value type (`Field<float,...>` vs `Field<double,...>`), and the codebase also contains mixed-precision examples where fields/particles deliberately use different precisions.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.30
- Local answer excerpt: Answer: IPPL supports mixed precision (float vs double) for fields through its utility functions and configuration options managed by the `Inform` class. Specifically, the `precision` method within the `Inform` class allows users to configure or retrieve th...

par_005 - How does IPPL handle multiple execution spaces like OpenMP and CUDA in the same build?

- Verdict: Partial
- Codex reference answer: Multiple execution spaces are enabled at build time through `IPPL_PLATFORMS` and Kokkos/HeFFTe backend flags; IPPL also has container helpers for multi-space data.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.39
- Local answer excerpt: Answer: IPPL handles multiple execution spaces like OpenMP and CUDA through the use of the `TypeForAllSpaces` struct defined in `Utility/TypeUtils.h`. This struct aggregates various memory and execution space types `ippl::variant<types (unique_memory_spaces...`

par_006 - What is the purpose of ParallelDispatch.h and what helpers does it expose?

- Verdict: Correct
- Codex reference answer: ParallelDispatch.h wraps Kokkos policies and functors, exposing RangePolicy, getRangePolicy, createRangePolicy, parallel_for, and parallel_reduce.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The ParallelDispatch.h file provides abstractions and utilities for parallel execution within the IPPL framework. It includes structs and functions that facilitate dispatching and managing of parallel tasks. Specifically, it exposes several key he...

par_007 - How does the Archive class serialize/deserialize Kokkos views for MPI transfer?

- Verdict: Partial
- Codex reference answer: Archive serializes Kokkos views into one-dimensional byte buffers with Kokkos kernels and memcpy, then deserializes them back into views on receive.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The retrieved context does not provide information on how the Archive class serializes/deserializes Kokkos views for MPI transfer. The context only includes namespace declarations related to MPI within the IPPL codebase. Evidence: - data/raw/ippl/sr...

par_008 - How are MPI tags managed in IPPL to avoid collisions across subsystems?

- Verdict: Incorrect
- Codex reference answer: MPI tags are managed with fixed base tags in Tags.h and the cyclic tag allocator in TagMaker, which Communicator inherits.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.29
- Local answer excerpt: Answer: The retrieved context does not provide specific information about how MPI tags are managed in IPPL to avoid collisions across subsystems. Evidence: - None of the provided chunks discuss MPI tags or their management in IPPL. The context covers variou...

par_009 - What does Kokkos::fence do, and when does IPPL call the wrapper ippl::fence?

- Verdict: Correct
- Codex reference answer: Kokkos::fence() waits for outstanding kernels; IPPL exposes ippl::fence() as a runtime wrapper in Ippl.cpp and also fences internally after many kernel/data-movement steps.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.29
- Local answer excerpt: Answer: Kokkos::fence is a synchronization operation that ensures all preceding operations on devices (like CPUs or GPUs) are completed before any subsequent operations begin. In IPPL, the wrapper ippl::fence calls MPI_Win_fence to synchronize remot...

Boundary And Halo

bc_001 - What boundary condition types are provided in BcTypes.h?

- Verdict: Correct

- Codex reference answer: BcTypes.h provides periodic, constant, zero, extrapolating, and no-BC face classes.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.70
- Local answer excerpt: Answer: The boundary condition types provided in BcTypes.h include: - ConstantFace: Applies constant value boundary conditions. - ExtrapolateFace: Extrapolates values at the boundaries. - NoBcFace: No boundary condition; does nothing specific. - PeriodicF...

bc_002 - How does a periodic boundary condition differ from a Neumann boundary condition in IPPL Fields?

- Verdict: Incorrect
- Codex reference answer: Periodic BCs wrap values from the opposite side of the domain; there is no explicit NeumannFace class in this codebase, so the closest non-periodic behavior is constant/extrapolating face treatment.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.24
- Local answer excerpt: The provided context does not contain any information about how periodic boundary conditions differ from Neumann boundary conditions in IPPL Fields. The chunks discuss initialization, setting periodic boundary conditions, and applying boundary conditions, b...

bc_003 - How are ghost cells populated during a halo exchange?

- Verdict: Partial
- Codex reference answer: Ghost cells are populated mainly by fillHalo() / PeriodicFace::apply(): source slabs are packed, exchanged, and unpacked into ghost regions; accumulateHalo_noghost() is a different operation.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.32
- Local answer excerpt: Answer: Ghost cells are populated during a halo exchange through the exchangeBoundaries method within the HaloCells<T, Dim, ViewArgs...> class. This process involves identifying neighboring cells that require data via FieldLayout::getNeighborsSendRange...

bc_004 - How are physical-cell values copied into ghost cells for periodic boundary conditions?

- Verdict: Correct
- Codex reference answer: Periodic ghost cells are filled from opposite-side physical cells in applyPeriodicSerialDim() and the multi-rank PeriodicFace::apply() path.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.37
- Local answer excerpt: Answer: Physical-cell values are copied into ghost cells for periodic boundary conditions through the setPeriodicBoundaryConditions method. This method initializes a vector of shared pointers to PeriodicFace objects for each face of the simulation grid ...

bc_005 - How does the absorbing boundary condition in the MaxwellSolvers module damp outgoing waves?

- Verdict: Correct
- Codex reference answer: The absorbing Maxwell BC uses second-order Mur-style formulas with precomputed weights and previous/current/next boundary values plus tangential neighbors to damp

outgoing waves.

- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.46
- Local answer excerpt: Answer: The MaxwellSolvers module defines structs and methods related to second-order absorbing boundary conditions (ABC) in the file AbsorbingBC.h. These boundary conditions are applied to fields within the simulation to damp outgoing waves. The AB...

Numerical Meaning

num_001 - What does the FFT-based Poisson solver physically compute in terms of Coulomb's law?

- Verdict: Incorrect
- Codex reference answer: The FFT Poisson solvers compute electrostatic potential (and optionally field/gradient) from charge density, i.e. the discretized Coulomb potential corresponding to $-\Delta\phi = \rho$.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.31
- Local answer excerpt: The provided retrieved context does not contain specific details about an FFT-based Poisson solver or its implementation. The chunks primarily discuss classes and methods related to solving Poisson's equation but do not mention FFTs or how they are applied....

num_002 - What is the physical meaning of the fields advanced by the FDTD solver?

- Verdict: Correct
- Codex reference answer: The Maxwell solvers evolve electromagnetic field data: electric and magnetic fields (E, B) driven by a source/four-current field.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.43
- Local answer excerpt: Answer: The FDTD solver advances the electromagnetic fields A_{np1} using a non-standard finite-difference time-domain method. It updates these fields by combining previous (A_{nm1}) and current (A_n) field values with source terms, employing anisotropy c...

num_007 - What are the numerical tradeoffs between the FFT and CG Poisson solvers in IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT Poisson is fast and direct on structured uniform grids with compatible BCs but less flexible; CG/PCG is iterative and may be slower/convergence-limited, but it is more flexible for alternative discretizations, BCs, and preconditioning.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.17
- Local answer excerpt: Answer: The retrieved context does not provide specific information about the numerical tradeoffs between the FFT and CG Poisson solvers in IPPL. The chunks discuss the Conjugate Gradient (CG) solver and Fast Fourier Transform (FFT) classes separately but d...

Build And Install

build_001 - Which third-party libraries does IPPL depend on, and what are they used for?

- Verdict: Correct
- Codex reference answer: Core third-party dependencies are MPI for distributed communication, Kokkos for performance portability, heFFTe for FFT backends/plans, FFTW/CuFFT as optional FFT backends, and GoogleTest for tests.

- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.46
- Local answer excerpt: Answer: IPPL depends on the following third-party libraries: 1. Kokkos: A performance-portable programming model that enables parallel execution across CPU and GPU architectures. It is used for managing parallel computations within IPPL. 2. **MPI (Messa...

build_002 - What C++ standard does IPPL require, and why?

- Verdict: Incorrect
- Codex reference answer: IPPL requires C++20, as stated in the README badge and the installation examples/configuration flags.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.00
- Local answer excerpt: Answer: The retrieved context does not specify the C++ standard required by the IPPL codebase. The provided chunks discuss initialization functions, utility classes, and parallel execution policies but do not mention any specific C++ standard requirements. ...

build_003 - How is IPPL configured with CMake to enable a GPU build?

- Verdict: Incorrect
- Codex reference answer: GPU builds are enabled through CMake options like IPPL_PLATFORMS=CUDA or IPPL_PLATFORMS="HIP;OPENMP" plus matching Kokkos/architecture/backend flags (Kokkos_ARCH_*, CMAKE_CUDA_ARCHITECTURES, Kokkos_ENABLE_HIP, HefftE_ENABLE_ROCM, etc.).
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.13
- Local answer excerpt: Answer: The retrieved context does not provide specific information on how to configure IPPL with CMake to enable a GPU build. The provided chunks focus on initialization functions, variant type handling, and utility functions, but do not include details ab...

build_004 - What options does INSTALLATION.md describe for configuring Kokkos for IPPL?

- Verdict: Correct
- Codex reference answer: INSTALLATION.md describes Kokkos version selection plus backend/architecture options used with IPPL builds, including CUDA and HIP/OpenMP examples and IPPL_PLATFORMS combinations.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: The INSTALLATION.md file describes several CMake options for configuring Kokkos in the context of building IPPL. Specifically, it mentions: - Kokkos_VERSION: The default version is 4.5.00. Users can specify a different version using the format...

build_005 - How do I install IPPL on the EULER cluster according to the documentation?

- Verdict: Correct
- Codex reference answer: The EULER install guide says to connect with VPN/SSH, switch to the new software stack, purge modules, load GCC/CMake/CUDA/OpenMPI, clone the repo, configure a build, compile, and test on an interactive node.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.63
- Local answer excerpt: Answer: To install IPPL on the EULER cluster according to the documentation: 1. Connect to the EULER cluster via SSH with X11 forwarding enabled: ssh -Y <username>@euler.ethz.ch. 2. Transition to the new software stack: env2lmod. 3. Ensure no previous m...

Testing And Workflow

test_002 - What naming conventions does WORKFLOW.md require for variables, constants, and member variables?

- Verdict: Correct
- Codex reference answer: WORKFLOW.md requires camelCase variables, CAPITAL_CASE compile-time constants, and member names suffixed with _m.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.75
- Local answer excerpt: Answer: WORKFLOW.md requires specific naming conventions for variables, constants, and member variables within the IPPL codebase. Variables should use camel casing (myVariable). Compile-time constants should use capital casing (MY_CONSTANT). Member vari...

test_003 - When should code use Kokkos::sqrt versus std::sqrt according to WORKFLOW.md?

- Verdict: Correct
- Codex reference answer: Use std::sqrt in host-only code, and Kokkos::sqrt in device-capable code such as KOKKOS_INLINE_FUNCTION kernels.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.75
- Local answer excerpt: Answer: According to WORKFLOW.md, code should use Kokkos::sqrt for device-code or potentially device-executed code (such as those marked with KOKKOS_INLINE_FUNCTION). For host-only code, std::sqrt from the standard library should be used. Evidence: ...

test_004 - How does IPPL profile MPI calls using mpiP?

- Verdict: Incorrect
- Codex reference answer: IPPL profiles MPI calls with mpiP by preloading libmpiP.so, optionally setting MPIP flags, then parsing the produced .mpiP output with the bundled scripts/mpiP.py.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.19
- Local answer excerpt: Answer: The provided context does not contain information on how IPPL profiles MPI calls using mpiP. The context includes descriptions of various files and classes related to initialization, communication, error handling, and type definitions, but there is ...

test_005 - What does the SLURM job template for Gwendolen in the README look like?

- Verdict: Incorrect
- Codex reference answer: The README does include a Gwendolen SLURM template showing the gmerlin6 cluster, gwendolen partition, GPU count, and srun ./<your_executable> <args> --kokkos-map-device-id-by=mpi_rank.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.17
- Local answer excerpt: Answer: The retrieved context does not provide specific information about a SLURM job template for Gwendolen. The content only mentions that SLURM Job scripts are discussed under a section in the README but does not include the actual template. Evidence: - ...